IIT Tirupati Navavishkār I-Hub Foundation

Geo-Intel Lab

AI/ML Hackathon - Ministry of Panchayati Raj (Geospatial Intelligence Challenge)

Powered by: ***Geo-Intel Lab, IITTNiF***

Problem Statement Addressed

DTM Creation using AI/ML from point cloud data and development of drainage network

Project Title - GramDrain

Name

Team Leader :- Monty Milan Biswal

Institution Name

SRM Institute of Technology, Kattankulathur

IIT Madras

Supported by

Geospatial Intelligence and Applications Laboratory,

IIT Tirupati Navavishkar I-Hub Foundation

1. Title of the Idea

GramDrain: AI-Powered Waterlogging Prediction and Resilient Drainage Network Design for Rural India

2. Team Name – Frustrated Founders

Monty Milan Biswal – SRMIST, Kattankulathur
9938729203 – mb4529@srmist.edu.in
Chinmay Mishra – SRMIST, Kattankulathur
9911478899 – cm1372@srmist.edu.in
Sneha Pathak – SRMIST, Kattankulathur
6201713468 – sp8364@srmist.edu.in
Shikhar Mohan – SRMIST, Kattankulathur
9971053261 – sm7308@srmist.edu.in
Bivesh Dalai – IIT Madras
8101476952 – ch24b046@smail.iitm.ac.in

3. Problem Statement Addressed

Rural villages across India face recurring waterlogging during monsoon seasons, damaging infrastructure, disrupting livelihoods, and causing public health crises. Gram Panchayats lack the technical tools to identify high-risk zones or plan drainage improvements scientifically. Existing approaches rely on manual surveys, which are time-consuming, expensive, and inconsistent across geographies. The challenge is to process high-density LiDAR point cloud data from diverse Indian villages - ranging from flat coastal settlements to hilly Punjab terrain - and automatically identify waterlogging hotspots and design optimised drainage networks, all without requiring specialised GIS expertise from local administrators.

4. Proposed Solution

The proposed system is a fully automated, end-to-end AI/ML pipeline that transforms raw aerial LiDAR point cloud data (.las/.laz) into actionable drainage network designs for rural Gram Panchayats.

Stage 1 - Ground Classification: Raw point clouds (up to 1.65 billion points per village) are processed using a memory-efficient chunked NumPy Grid Accumulator combined with the Cloth Simulation Filter (CSF) algorithm to extract bare-earth ground points from

vegetation, buildings, and other above-ground features. A dynamic memory scaling engine automatically computes optimal voxel resolution and stride parameters based on file metadata.

Stage 2 - DTM Generation: A 1 m resolution Digital Terrain Model (OGC-compliant Cloud-Optimised GeoTIFF) is interpolated from classified ground points using a weighted uniform filter with Gaussian smoothing to fill data voids.

Stage 3 - Hydrological Feature Extraction: A 12-feature XGBoost classifier trained on terrain derivatives (slope, curvature, flow accumulation, topographic wetness index, and multi-scale relief) identifies waterlogging hotspot zones. Flow accumulation is computed using a multi-scale minimum-filter gravity approximation to delineate stream channels.

Stage 4 - Drainage Network Design: A gravity-corrected Dijkstra routing algorithm connects waterlogging hotspot centroids to the nearest natural drainage axis via least-cost paths, ensuring all proposed drains are gravity-fed and follow existing valley axes. The output is a GeoPackage of proposed drainage lines with geometry, length, and cluster attributes ready for Gram Panchayat implementation.

Stage 5 - Interactive Dashboard: A Streamlit-based web dashboard enables non-technical Gram Panchayat users to upload LiDAR files and receive a full parameter sheet and pipeline outputs without writing any code.

5. Uniqueness and Innovation

- **Memory-Adaptive Processing at Scale:** Existing LiDAR processing tools (LAStools, PDAL) require manual parameter tuning per file and crash on billion-point datasets without expensive hardware. Our dynamic scaling engine automatically adjusts voxel resolution and sampling stride based on point count and spatial extent, replacing a Python dictionary accumulator (which consumed 4.9 GB for a 32M-cell grid) with a NumPy pre-allocated grid accumulator (305 MB for the same data) - a 16X RAM reduction enabling 1.65-billion-point files on standard Colab infrastructure.
- **Gravity-Aware Drainage Routing:** Unlike rule-based flow accumulation (D8/D-infinity), our Dijkstra cost function combines normalized flow accumulation (60%) with normalized elevation (40%), guaranteeing that every proposed drain is physically realizable, downslope, aligned with natural valleys, and never crossing NODATA (village boundary) cells.
- **Multi-Scale Terrain Intelligence:** The 12-feature XGBoost model incorporates relief at 5, 21, and 51 - pixel radii simultaneously, capturing both micro-topographic

depressions (inundation pockets) and macro-scale valley structure, something single-scale slope thresholding cannot achieve.

- **OGC-Compliant Outputs:** All rasters are exported as Cloud-Optimised GeoTIFFs (COGs) and vector outputs as GeoPackages, making them directly consumable by government GIS platforms (SVAMITVA, BhuNaksha) without format conversion.

Unlike existing tools that require manual GIS input or are limited to low-resolution satellite-derived DEMs, GramDrain operates directly on centimetre-accuracy LiDAR point clouds. directly maps to the deliverables specified by the Ministry of Panchayati Raj, requiring zero manual post-processing.

6. Technology Stack / Methodology

Stage 1 - LiDAR Ingestion & Memory-Adaptive Ground Classification

1a. File I/O and Coordinate Type Detection

Raw point cloud files are opened in streaming mode without loading all points into RAM. The LAS header is read first to extract:
- *point_count* (9.8M to 1.65B points across 10 villages)
- Bounding box '[xmin, xmax, ymin, ymax]'
- Coordinate type auto-detection: if *max(x_range, y_range) < 2.0*, coordinates are **geographic (WGS84/EPSG:4326)**; otherwise **projected (UTM)**.

This distinction drives reprojection logic downstream.

1b. Dynamic Memory Scaling

Before reading any points, the system computes two critical parameters from the header metadata alone:

- **VOXEL_RES** - the horizontal voxel size in native coordinate units. The spatial range is first converted to metres (with a latitude-aware cosine correction for geographic files):
  - 'x_range_m = x_range × 111,000 × cos(lat_mid)' for geographic files
  - 'min_voxel_m = max(x_range_m, y_range_m) / 4,000' - ensures the grid never exceeds 4,000 × 4,000 = 16M cells
  - The final voxel size is snapped to the nearest value from the candidate list: `[0.05, 0.10, 0.15, 0.20, 0.30, 0.50, 0.75, 1.00, 1.50, 2.00, 3.00, 5.00, 10.00]` metres
- **STRIDE** - the point sub-sampling stride (take every Nth point). Computed as:
  - 'stride_dict = [worst_cells / 33,000,000]' - caps occupancy to 33M entries (heap RAM constraint)
  - 'stride_pts = [total_pts / 330,000,000]' - caps in-flight GC pressure for 1B+ point files

- 'STRIDE = min(max(stride_dict, stride_pts), 50)'

This auto-scaling ensures *zero manual intervention*: KADAMTALA_RNG (1.65B points, 6.2 km × 4.1 km) is processed with the same code path as CHAKHIRASINGH (9.8M points, 910 m × 680 m).

1c. NumPy Grid Accumulator (min-z voxelisation)

The file is read in **2-million-point chunks** using `laspy.chunk_iterator`. For each chunk:

- Stride decimation
- Cell index computation
- Intra-chunk deduplication (fully vectorised):
- Cross-chunk grid update

Three pre-allocated arrays constitute the entire accumulator - 305 MB maximum for a 4,000 × 4,000 grid, versus ~4.9 GB for an equivalent Python dict

Result: a dense "(N, 3)" NumPy array of minimum-z representatives, one per occupied voxel, via a single extraction.

1d. Geographic → UTM Reprojection

For geographic-coordinate files (PIRAYANKUPPAM, THANDALAM), *pyproj.Transformer* reprojects the downsampled ground candidates from EPSG:4326 to the target UTM zone (EPSG:32644) before CSF. This is done after downsampling, not on the full point cloud, for efficiency.

1e. Cloth Simulation Filter (CSF) Ground Classification

The downsampled, projected point set (≤5M points) is passed to the Cloth Simulation Filter - a physics-based algorithm that simulates a cloth draped over the inverted point cloud:

Outputs: gnd_idx (ground point indices) and off_idx (above-ground). Ground points are saved as LAS 1.2 point-format-2 files with classification code 2 (standard bare-earth).

Stage 2 - Digital Terrain Model (DTM) Generation

Ground points are rasterised to a 1 m resolution float32 grid (EPSG-correct UTM CRS):

1. Min-z binning: each ground point is assigned to its nearest grid cell; where multiple points fall in one cell, the minimum z is retained

2. Void filling: cells with no data are filled using a weighted uniform filter (5 × 5 kernel), the weight is the valid-data mask, so boundary effects at data voids are handled gracefully:
dtm = uniform_filter(dtm_f, 5) / max(uniform_filter(valid_mask, 5), 1e-6)
3. Noise suppression: *median_filter(size=3)* removes outlier spikes from residual non-ground misclassifications
4. Smooth interpolation: *gaussian_filter(sigma=1)* provides sub-pixel continuity for CSF-derived DTMs that may have slight cloth-resolution artefacts

Output: OGC Cloud-Optimised GeoTIFF (COG) with LZW compression, 256 × 256 tiling, predictor=2, and NODATA=-9999, directly compatible with QGIS, ArcGIS, and government GIS portals (BhuNaksha, SVAMITVA).

Stage 3 - Terrain Derivative Computation (Slope, Flow Accumulation, TWI)

3a. Slope
Primary: WhiteboxTools *slope()* function (Horn's 3 × 3 finite-difference method) in degrees. Fallback (if WBT unavailable): np.gradient + arctan(sqrt($dx^2 + dy^2$)) in degrees.

3b. Multi-Scale Flow Accumulation
A computationally efficient gravity-proxy accumulation is computed using three nested minimum filters at radii 5, 21, and 51 pixels:

r5 = DTM - minimum_filter(DTM, 5) → micro-depression depth
r21 = DTM - minimum_filter(DTM, 21) → valley-scale depression
r51 = DTM - minimum_filter(DTM, 51) → macro-drainage basin
accum = 0.2·exp(−r5/σ5) + 0.3·exp(−r21/σ21) + 0.5·exp(−r51/σ51)

Each term is an exponential decay: cells at the bottom of depressions (small relative relief) score close to 1.0. The weighted sum emphasises macro-valley structure (50% weight) while preserving micro-pond signatures. A final *gaussian_filter(sigma=3)* suppresses salt-and-pepper noise. Stream channels are identified as the top 3rd percentile of accumulation values.

3c. Topographic Wetness Index (TWI)
TWI = ln((accum × 100 + 1) / (tan(slope_rad) + 1e-6))
High TWI indicates flat, low-lying areas with concentrated upslope drainage - the primary physical indicator of waterlogging susceptibility in rural terrain.

Stage 4 - Waterlogging Hotspot Classification (Rule-Based Labels + XGBoost ML)

4a. 12-Feature Matrix

For every pixel of the DTM, 12 terrain features are computed:

	Feature	Window	Physical meaning
1	Elevation (z)	Point	Absolute height above datum
2	Slope (°)	Point	Local steepness
3	Flow accumulation	Point	Upslope drainage concentration
4	TWI	Point	Wetness potential
5	Laplacian curvature	Point	Concavity / convexity
6	Distance to outlet	Point	Proximity to lowest point
7	Mean elevation	5 × 5	Neighbourhood elevation context
8	Std deviation of elevation	5 × 5	Micro-roughness
9	Mean slope	5 × 5	Neighbourhood gradient
10	Mean TWI	5 × 5	Neighbourhood wetness
11	Mean flow accumulation	11 × 11	Meso-scale drainage density
12	Elevation range	11 × 11	Local relief (depression depth)

4b. Rule-Based Training Labels

Ground truth labels for XGBoost training are generated from the terrain derivatives themselves using a physically motivated weighted rule:

risk_rule = 0.40·norm(TWI) + 0.35·norm(accum) + 0.15·(1 − norm(slope)) + 0.10·(1 − norm(z))

Thresholds:

- risk_rule > 0.92 → class 2 (High risk)
- 0.78 ≤ risk_rule ≤ 0.92 → class 1 (Medium risk)
- < 0.78 → class 0 (Safe / dry)

These labels serve as pseudo-ground-truth for XGBoost training, enabling the ML model to generalise beyond simple thresholding.

4c. XGBoost Classifier

The XGBoost classifier is retrained per village on up to 200,000 randomly sampled pixels, ensuring it adapts to each village's terrain character:

Features are StandardScaler-normalised before training. Prediction is done in 500,000-pixel chunks to prevent RAM exhaustion on large rasters.
The confidence map (*predict_proba[:, 2]*) for the High-risk class is exported as a separate COG for uncertainty-aware planning.

4d. Morphological Post-Processing
Raw ML predictions are refined using binary morphological operations:
- *3 × 3 binary opening*: removes isolated single-pixel noise (false positives from sensor noise or classification artefacts)
- *5 × 5 binary closing*: fills small holes within genuine hotspot clusters, producing contiguous waterlogging zones suitable for civil works planning
Applied per class (High first, then Medium) with NODATA masking to prevent boundary artefacts. Fallback: if total ML-detected cells < 10 (model failure / featureless terrain), the rule-based labels are used directly.

Stage 5 - Gravity-Corrected Drainage Network Design

5a. Cost Surface
A composite cost raster combines normalised flow accumulation and normalised elevation:
cost[r,c] = (1 − norm_accum[r,c]) × 60 + norm_elev[r,c] × 40 + 1
- *Low cost* at valley floors (high accumulation + low elevation) → drains follow natural watercourses
- *+1 base cost* ensures no zero-cost cells (prevents degenerate shortest paths)
- *NODATA cells* assigned cost = 9,999 → drains never exit the village boundary

5b. Hotspot Cluster Detection
High-risk pixels are labelled into connected components using *scipy.ndimage.label*. Each component with ≥ 25 pixels (≥ 25 m^2 at 1 m resolution) is eligible for drain routing. Up to 50 clusters are routed per village.

5c. Dijkstra Routing (8-connectivity)
For each hotspot cluster, its centre of mass is computed as the routing source. A priority-queue Dijkstra finds the minimum-cost path from the hotspot centroid to the nearest stream-channel cell (top 3rd percentile of flow accumulation), using 8-directional movement (diagonal cost = 1.414 × cell cost):
The resulting pixel path is converted to real-world coordinates via the raster affine transform and stored as a Shapely LineString.

5d. Vector Outputs

All drainage features are exported as an OGC GeoPackage (.gpkg) with three layers:

- proposed_drains - geometry (LineString), cluster_id, hotspot_area (m^2), length_m, village
- hotspots - geometry (Polygon), risk_level (High/Medium), risk_code, area_m2, village
- streams - geometry (Polygon), feature_type, physics_valid, village

TECHONOLOGY STACK

Component	Library / Version	Function
LiDAR I/O	laspy[lazrs]	.las/.laz streaming, chunked iteration
Ground classification	cloth-simulation-filter (CSF)	Physics-based bare-earth extraction
Raster I/O & CRS	rasterio 1.5.0	GeoTIFF read/write, COG export, transform
Vector operations	geopandas 1.1.3, shapely	GeoPackage export, LineString construction
ML classification	xgboost 3.2.0, scikit-learn	3-class hotspot prediction, StandardScaler
Terrain derivatives	scipy.ndimage, whitebox	Filters, slope, morphology
Coordinate transforms	pyproj	WGS84 → UTM reprojection
Routing	heapq, networkx	Dijkstra shortest-path drainage routing
Numerical core	numpy 2.0.2	Grid accumulation, feature matrix, masking
Visualisation	matplotlib	Per-village PNG reports (DTM + risk + drains)
Runtime	Google Colab (High-RAM)	≥52 GB RAM, Google Drive persistent storage

7. Expected Impact

Environmental: Systematic drainage of waterlogged zones reduces standing water duration, lowering mosquito breeding habitats and associated disease burden (malaria, dengue). Improved drainage reduces soil salinization and waterlogging damage to agricultural land, preserving cultivable area.

Social: Gram Panchayats gain a data-driven drainage plan within hours instead of months of manual survey. Rural communities in flood-prone areas, particularly coastal Andaman villages and flat Punjab plains all benefit from reduced post-monsoon inundation that currently disrupts livelihoods and mobility.

Economic: Gravity-fed drain routing minimizes construction length and eliminates pumping requirements, reducing civil works cost by an estimated 30–45% versus ad-hoc drain placement. Protecting agricultural land from waterlogging directly preserves crop yields.

Governance: OGC-compliant COG and GeoPackage outputs integrate directly with SVAMITVA, BhuNaksha, and state GIS portals, enabling Panchayat officials to view, validate, and share drainage plans without specialist GIS software. A Streamlit web dashboard allows Gram Panchayat officials to upload LiDAR data and receive drainage plans interactively - eliminating the need for command-line access or GIS expertise entirely.

Scalability Signal: The pipeline processed 10 villages spanning 9 million to 1.65 billion points (11.6 GB single file) on a single Colab session, confirming readiness for district-scale deployment.

8. Implementation Plan / Roadmap

Month 1 (Foundation): *(COMPLETED)*

- Finalize pipeline robustness across all 10 pilot villages
- Integrate WhiteboxTools flow direction (D8) as primary accumulation method with NumPy fallback
- Validate DTM accuracy against field benchmark points (where available)
- Build automated report generation (PDF + JSON) per village

Month 2-3 (Validation & Ground Truth):

- Collaborate with Gram Panchayat officials in 2–3 pilot villages to review proposed drainage lines
- Collect field feedback on hotspot accuracy and drain alignment
- Retrain XGBoost model with ground-truth waterlogging labels if available

Month 4 (Portal Integration):

- Develop lightweight Streamlit dashboard for non-technical Panchayat users to upload LiDAR data and view outputs *(COMPLETED)*
- Integrate COG outputs with BhuNaksha/SVAMITVA API endpoints
- Add multi-village batch processing queue

Month 5-6 (District Pilot):

- Expand to 50+ villages in one district (target: Punjab or Tamil Nadu)

- Implement automated cloud pipeline (GCP Cloud Run or AWS Lambda) for on-demand processing
- Performance benchmarking and cost optimization

Month 7–9 (National Scale):

- Onboard 3 additional states; localize for UTM zones across all Indian regions
- Publish methodology as open standard; submit for SVAMITVA integration proposal
- Training workshops for District Collectorate GIS teams

9. Required Resources

Compute:

- High-RAM cloud instances (≥52 GB RAM) for billion-point files (Google Colab Pro+ or GCP n2-highmem-8)
- GPU optional (XGBoost training only); CPU sufficient for inference

Data:

- Aerial LiDAR point clouds (.las/.laz) from SVAMITVA drone surveys, already available for pilot villages
- Village boundary shapefiles (GeoPackage) for NODATA masking
- Ground-truth waterlogging incident records from district disaster management authorities for model validation.

Software (all open-source / free tier):

- Python ecosystem (laspy, rasterio, CSF, XGBoost, GeoPandas, WhiteboxTools, streamlit)
- Google Colab Pro+ for development; GCP/AWS for production deployment

Human Resources:

- 2 ML/GIS engineers for pipeline maintenance and model retraining
- 1 field validation coordinator for Gram Panchayat liaison
- Domain support from IITTNiF Geospatial Intelligence Lab

Estimated cloud cost at scale: ~₹8–12 per village for processing (including storage), making district-scale deployment (500 villages) feasible within ₹6,000 compute budget.

10. Scalability and Sustainability

Scalability: The pipeline is stateless and parameterless, all processing parameters (voxel resolution, stride, UTM zone) are auto-computed from LAS file metadata. Adding a new village requires only registering its file path and EPSG code. The chunked architecture handles any point count without code changes; the dynamic scaling engine has been validated from 9M to 1.65B points.

Horizontal scaling is trivial: each village is an independent processing job, enabling parallel cloud execution across hundreds of villages simultaneously using managed services (GCP Cloud Run, AWS Batch).

Sustainability: The system relies entirely on open-source libraries with no licensing fees. Aerial LiDAR data collection is already funded and operational under the SVAMITVA scheme, eliminating data acquisition costs. Maintenance effort is minimal once deployed - only model retraining is periodic (annually or when new ground-truth labels are collected).

Long-term, the OGC-compliant output format ensures compatibility with future government GIS upgrades. The system can be extended to additional use cases (road network extraction, building footprint detection, land use classification) using the same DTM and point cloud infrastructure, multiplying value per village

11. Stakeholders and Collaborations (if any)

Primary Stakeholder: Ministry of Panchayati Raj (MoPR), the drainage plans directly support Gram Panchayat Development Plans (GPDP) under the XIV/XV Finance Commission and MGNREGS drainage works.

Data Provider: Survey of India / SVAMITVA scheme, source of aerial LiDAR point clouds for pilot villages across 10 states.

Technical Host: Geospatial Intelligence and Applications Laboratory, IIT Tirupati Navavishkar I-Hub Foundation, provides research infrastructure, validation expertise, and academic credibility.

Implementation Partners:

- District Collectorate GIS Cells - for field validation and Panchayat-level adoption
- State Remote Sensing Application Centres (SRSACs) - for integration with state GIS portals
- National Informatics Centre (NIC) - for BhuNaksha/eSvamitva API integration

Community: Gram Panchayat officials and Panchayat Raj Institution (PRI) members are the end-users; their feedback loop via the Streamlit dashboard ensures the system remains field-relevant

12. Current Stage of Development

Stage: Prototype

A fully functional end-to-end prototype has been implemented and tested across 10 Gram Panchayat villages from 5 states (Punjab, Tamil Nadu, Gujarat, Andaman & Nicobar). The pipeline successfully processes point clouds ranging from 9.8 million to 1.65 billion points, generates 1 m DTMs, classifies waterlogging hotspots using a 12-feature XGBoost

model, and produces gravity-fed drainage network proposals exported as OGC-compliant GeoPackages and COG GeoTIFFs.
Key prototype achievements:

- 10/10 villages fully processed without manual intervention
- Memory crash on 1.65B-point file resolved via NumPy Grid Accumulator (16× RAM reduction)
- Per-village JSON/visual reports generated with drainage length, hotspot count, and area statistics
- Validated on diverse terrain types: flat irrigated plains (Punjab), coastal lowlands (Andaman), peri-urban areas (Tamil Nadu)
- Interactive Streamlit dashboard integrated and verified locally

Next step: field validation of proposed drainage lines with Gram Panchayat engineers in 2 pilot villages.

13. Supporting Materials

- https://drive.google.com/drive/folders/1xAdnnwXTsU6IGPbTcUgwS_mB2KkjNGB5
  - ✓ mopr_outputs/dtm – Cloud-Optimised GeoTIFFs of Digital Terrain Models
  - ✓ mopr_outputs/gpkg – OGC GeoPackages (proposed drains, hotspots, streams)
  - ✓ mopr_outputs/ground – Classified LAS files (bare-earth points)
  - ✓ mopr_outputs/rasters – Slope, TWI, flow accumulation, risk, confidence COGs
  - ✓ mopr_outputs/village_reports – Per-village PNG reports
- GitHub Repository: https://github.com/SPACE-MONARCH/GRAM-DRAIN
  - ✓ docs/architecture_diagram.jpeg – system architecture overview
  - ✓ docs/DashboardExample_DHAL.pdf – Dashboard overview
  - ✓ docs/outputs_guide.md – Guide to check the outputs for the file
  - ✓ outputs/village_reports/ - per-village PNG reports (DTM + risk map + confusion matrix)
  - ✓ outputs/gpkg/ - OGC GeoPackages (3 layers per village)
  - ✓ outputs/dtm/ - Cloud-Optimised GeoTIFFs
  - ✓ notebook/IITT.ipynb - fully reproducible Google Colab pipeline